Task 2: From Requirement to Nested Structures (8 Marks)

Task 2.1: Identify the Entities (2 Marks)

Entity	Fields	Nested inside
Parcel (main entity)	id, weight, booking date, sender, receiver, service type, charge	none (it is the top-level structure)
Person (sender / receiver)	name, address	Parcel (used twice: sender and receiver)
Address	city, PIN code	Person
Date	day, month, year	Parcel (booking date)
Service type	NORMAL or EXPRESS (an enumeration)	Parcel

Nesting diagram

```
Parcel
 |-- id
 |-- weight
 |-- booked    : Date    { day, month, year }
 |-- sender    : Person  { name, addr : Address { city, pincode } }
 |-- receiver  : Person  { name, addr : Address { city, pincode } }
 |-- type      : enum service { NORMAL, EXPRESS }
 '-- charge
```

Task 2.2: Define the Structures (2 Marks)

```
#define MAX_PARCELS 100

enum service { NORMAL, EXPRESS };

typedef struct { int day, month, year; } Date;
typedef struct { char city[20]; int pincode; } Address;
typedef struct { char name[30]; Address addr; } Person;

typedef struct {
    int      id;
    float    weight;
    Date     booked;
    Person   sender;
    Person   receiver;
    enum service type;
    float    charge;
} Parcel;

Parcel bookings[MAX_PARCELS];
int    booking_count = 0;
```

Because Address is defined once and reused inside Person, and Person is used for both sender and receiver, no field is declared twice.

Task 2.3: Design the Operations (3 Marks)

(a) Sort the parcels by weight (selection sort, ascending)

```
SORT_BY_WEIGHT(P[0..n-1])
  for i <- 0 to n-2
      min <- i
      for j <- i+1 to n-1
          if P[j].weight < P[min].weight then min <- j
      if min != i then
          temp   <- P[i]          // whole structure copied
          P[i]   <- P[min]
          P[min] <- temp
```

Time: O(n^2) comparisons and at most n-1 structure swaps.

(b) List all parcels for a given receiver city

```
LIST_BY_CITY(P[0..n-1], city)
  found <- 0
  for i <- 0 to n-1
     if P[i].receiver.addr.city equals city      // strcmp(...) == 0
        print P[i].id, P[i].weight, P[i].receiver.name
        found <- found + 1
  if found = 0 then print "No parcels for ", city
```

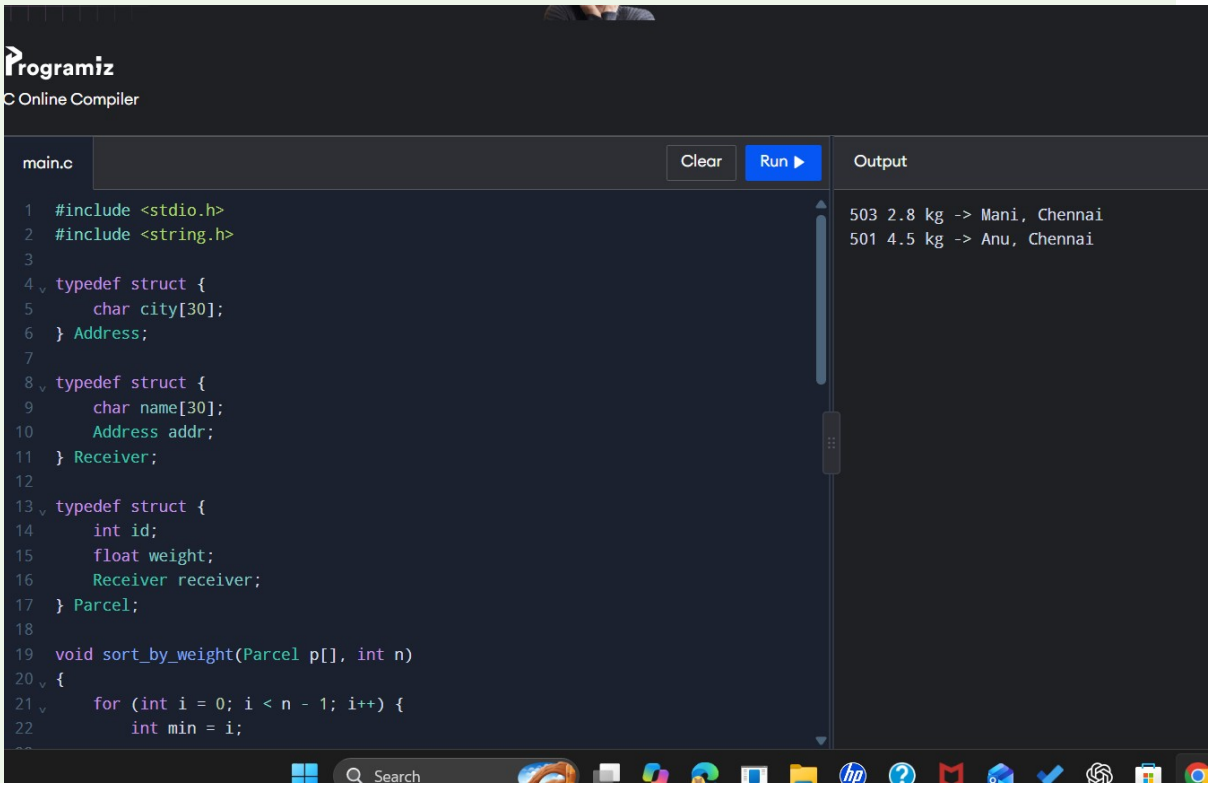
Time: O(n), one linear pass. A working C version of both operations (tested with three sample parcels):

```
void sort_by_weight(Parcel p[], int n)
{
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (p[j].weight < p[min].weight) min = j;
        if (min != i) { Parcel t = p[i]; p[i] = p[min]; p[min] = t; }
    }
}

void list_by_city(const Parcel p[], int n, const char *city)
{
    int found = 0;
    for (int i = 0; i < n; i++)
        if (strcmp(p[i].receiver.addr.city, city) == 0) {
            printf("%d %.1f kg -> %s, %s\n", p[i].id, p[i].weight,
                   p[i].receiver.name, p[i].receiver.addr.city);
            found++;
        }
    if (!found) printf("No parcels for %s\n", city);
}
```

Output

```
503 2.8 kg -> Mani, Chennai
501 4.5 kg -> Anu, Chennai
```



Task 2.4: Access Paths (1 Mark)

The PIN code of the receiver of the **third** parcel (index 2, since arrays start at 0):

Method	Expression
(i) Array and dot operator	bookings[2].receiver.addr.pincode
(ii) Structure pointer and arrow operator	Parcel *p = bookings; then (p + 2)->receiver.addr.pincode (equivalent: p[2].receiver.addr.pincode or (&bookings[2])->receiver.addr.pincode)

The arrow is used only once, for the pointer. After that, receiver and addr are structures, not pointers, so the dot operator is used. Both expressions printed 600077 in the test.

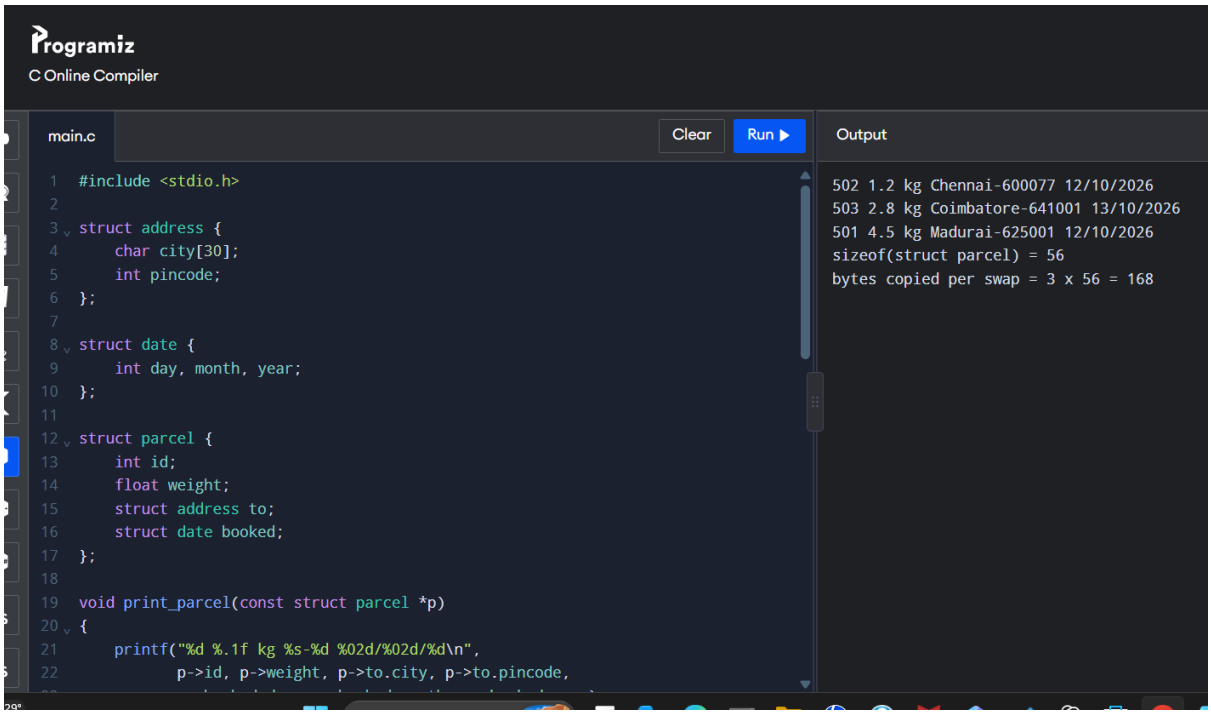
Complete the Code: TODO 1 and TODO 2

```
void print_parcel(const struct parcel *p)
{
    printf("%d %.1f kg %s-%d %02d/%02d/%d\n",
           p->id, p->weight, p->to.city, p->to.pincode,
           p->booked.day, p->booked.month, p->booked.year);
}

void sort_by_weight(struct parcel p[], int n)
{
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (p[j].weight < p[min].weight) min = j;
        if (min != i) {
            struct parcel tmp = p[i];
            p[i] = p[min];
            p[min] = tmp;
        }
    }
}
```

Output

```
502 1.2 kg Chennai-600077 12/10/2026
503 2.8 kg Coimbatore-641001 13/10/2026
501 4.5 kg Madurai-625001 12/10/2026
sizeof(struct parcel) = 44, bytes copied per swap = 3 x 44 = 132
```



Bytes copied per swap. sizeof(struct parcel) = 44 bytes (4 id + 4 weight + 24 address + 12 date). A swap uses three structure assignments (tmp = a; a = b; b = tmp), so each swap copies 3 x 44 = **132 bytes**. This data needs 2 swaps, so 2 x 132 = 264 bytes are copied. In the worst case (n-1 swaps) the sort copies 132(n-1) bytes.

Alternative Concept and Complexity Analysis: Flat Structure

Flat version (address and date fields written directly inside the parcel)

```
struct parcel_flat {
    int   id;
    float weight;
    char  city[20];  int pincode;        /* receiver */
    int   day, month, year;              /* booking date */
};
```

	Nested structures	Flat structure
Access path (receiver PIN, parcel pointer p)	p->to.pincode (or p->receiver.addr.pincode)	p->pincode: shorter, but names must encode the role
Reuse of the address type	One struct address serves sender and receiver, and a function such as print_address(const struct address *) works on both	Fields must be repeated (s_city, s_pin, r_city, r_pin); no address-level function can be shared; a change to the city length must be made in several places
Passing a part	Can pass &p->to alone	Cannot pass "the address" as one unit
Run-time cost	Same: member offsets are fixed at compile time, so the machine code is the same	Same
Space (measured)	Exercise parcel: 44 bytes. Full parcel with sender, receiver, name, service and charge: 140 bytes	Exercise parcel: 44 bytes. Full flat parcel: 140 bytes

The space is identical because every member needs 4-byte alignment and no padding is inserted in either layout. Nesting costs nothing in memory or speed and gains reuse, clarity and safer passing of sub-records.